

• 바이브코딩 · PoC 개발 가이드

나의 첫 PoC, 세상에 꺼내는 법

Lovable 심화학습 : 버전관리·협업·배포 그리고 보안

2026.5.22. 개인정보 AX 엔진룸 / 내부 AI 연구모임

발표자 : 연구모임장 주문호

왜 오늘 이 주제를 다루나

PoC를 만들었다면, 이제 **관리**가 필요합니다.

1

최신 버전은 어디에 있지?

→ GitHub로 기록을 남깁니다.

2

다른 사람에게 어떻게 보여주지?

→ 배포로 공개 주소를 만듭니다.

3

공개하면 위험한게 있을까?

→ 보안으로 비밀을 지킵니다.

오늘의 지도

기록하고, 공개하고, 지키고, **배운다.**

PART 1

GitHub

버전관리와 협업체계

PART 2

배포

내 사이트를 공개하기

PART 3

보안

민감한 정보 관리

PART 4

시연 발제

PoC를 통해 조직의
AX 포인트 발견하기

PART 1

GitHub

— 버전관리와 협업

버전관리는 개발자만의 이야기가 아닙니다

우리는 이미 **버전관리**를 하고 있었어요.
다만, 체계적이지 않았을 뿐입니다.

내 PC ▶ 정책기획팀 ▶ 보고서

보고서_최종.hwp

보고서_최종_수정.hwp

보고서_최종_진짜최종.hwp

보고서_최종_진짜최종_국장님수정.hwp

보고서_최종_진짜진짜최종_제출용.hwp

이 방식의 문제

- ! 어느 것이 최신인지 헷갈린다.
- ! 누가 언제 무엇을 바꿨는지 모른다.
- ! 이전 버전이 필요할 때 찾기 어렵다.
- ! 여러 명이 동시에 수정하면 꼬인다.
- ! 파일명은 늘어나는데 변경 이유는 안 남는다.

GIT이란?

Git = 자동 변경이력 기록 시스템

파일 이름을 계속 바꾸지 않아도, 변경의 역사가 자동으로 남습니다.

Git이 자동으로 남기는 것

 언제 바뀌었는가 누가 바꿨는가 무엇이 바뀌었는가 왜 바꿨는가 필요하면 그 시점으로 돌아갈 수 있는가

한 줄 비유

보고서 파일의 '버전 기록'을
시스템으로 관리할 수 있도록한 것.

어려운 기술이 아니라,
'변경 이력을 자동으로 남기는 장치'.

GITHUB란?

GitHub = 클라우드 저장소 + 협업 공간

Git

내 작업물의 변경이력을 남기는 기능

GitHub

그 변경이력과 프로젝트 파일을
클라우드에 보관하고 함께 관리하는 공간

GitHub가 해주는 일



백업



소유권



협업



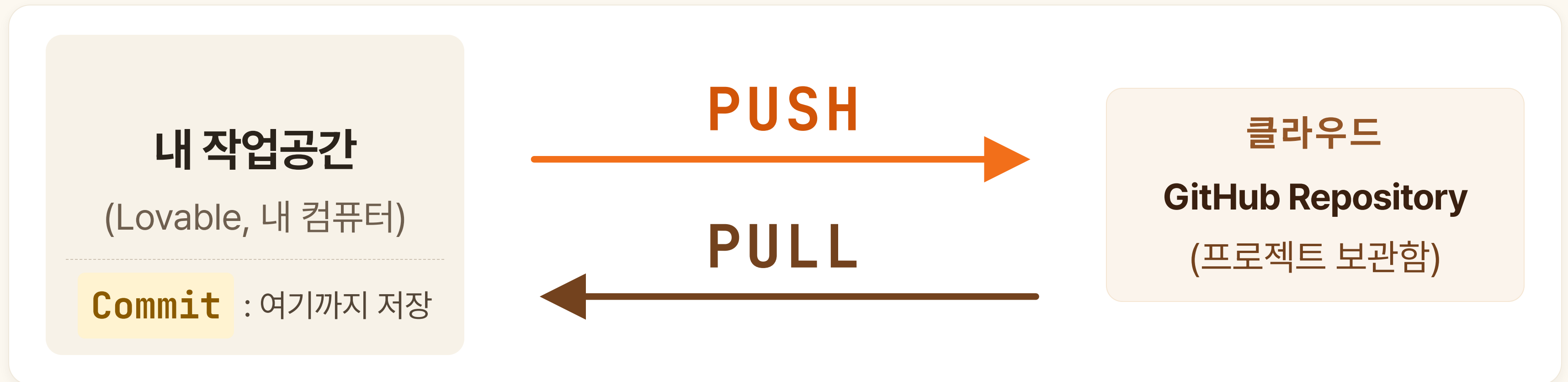
이력관리



확장성

핵심 용어 4개

Repository · Commit · Push · Pull



Repository

프로젝트 전용 보관함

Commit

의미 있는 저장점
(찰칵! 세이프 포인트)

Push

내 작업을 GitHub에 올리기

Pull

GitHub에서
최신 내용 가져오기

협업과 실험을 안전하게

Branch는 '원본을 건드리지 않는 평행 실험 공간'입니다.

Branch · PR · Merge



Clone

GitHub Repo



내 작업공간

Branch

원본을 건드리지 않는
별도 작업선(평행우주)

Merge

별도 작업선을
원본에 합치기

PR (Pull Request)

"합쳐도 될까요?" 검토요청

Clone

GitHub 프로젝트를
내 작업공간으로 복사해옴

개인이 GITHUB를 활용하는 방법

혼자 만드는 PoC에도 **GitHub**는 필요합니다.

• 시나리오 : 개인정보 자가점검 도구 혼자 만들기

- 1 Lovable로 첫 PoC 생성
- 2 GitHub에 프로젝트 연결
- 3 기본 화면 완성 후 Commit
- 4 새 기능 추가 전 Branch 생성
- 5 기능 완성 후 Merge
- 6 Publish로 공개

혼자 작업하는 사람의 핵심 습관 3가지

✓ 잘 되는 시점마다 **Commit**

화면이 멀쩡할 때마다 "여기까지 OK" 저장점 찍기.

✓ 새 기능 전 **Branch** 생성

실험은 평행우주에서. 본진은 안전하게.

✓ 망해도 **main**은 유지

실패해도 돌아갈 수 있는 '안전한 실험' 환경.

협업할 때 GITHUB는 왜 중요한가

여러 사람의 작업이 뒤섞이지 않게 도와줍니다.

• 시나리오 : 민원 분류 PoC를 3명이 함께하는 경우

A 팀원 A
화면 디자인 및 수정 (화면 다듬기)

B 팀원 B
민원 분류 기능 구현

C 팀원 C
민원 관련 법조항 추출 기능 구현

GitHub가 도와주는 것

- ✓ 누가 무엇을 바꿨는지 모두 기록
- ✓ 3명이 동시에 작업 가능
- ✓ 검토(PR) 후 반영 가능
- ✓ 덮어쓰기 사고 자동 방지

LOVABLE  GITHUB

Lovable은 만들고, GitHub는 기록합니다.

Lovable에서 하는 일	GitHub 관점
채팅으로 수정 요청	→ 변경사항 생성
수정 결과 반영	→ Commit 생성
GitHub 연결	→ Repository 생성 및 연결
수정 저장	→ main 브랜치에 Push
Publish	→ 현재 앱의 공개 URL 생성

PART 2

배포 — 내 앱을 세상에

내 화면에서는 되는데, 왜 다른 사람은 못 볼까?

LOCALHOST와 공개 URL

내 화면에서는 되는데, 왜 다른 사람은 못 볼까?

Preview · 시제품 모드

내 작업공간 안에서만 실행 중
(127.0.0.1) (localhost : 3000)

- 안전하게 실험 가능
- 나만의 미리보기 화면
- × 다른 사람은 접근 불가

배포된 앱 · 공개 모드

- 공개 URL 존재
- 링크 한 번이면 접속
- ✓ 다른 사람도 실제로 사용 가능

배포란?

배포 = 내가 개발한 앱에 **공식 주소**를 만드는 일

1

서버

앱을 계속 켜두는
인터넷상의 컴퓨터

2

배포

앱을 인터넷에서
볼 수 있게 올리는 과정

3

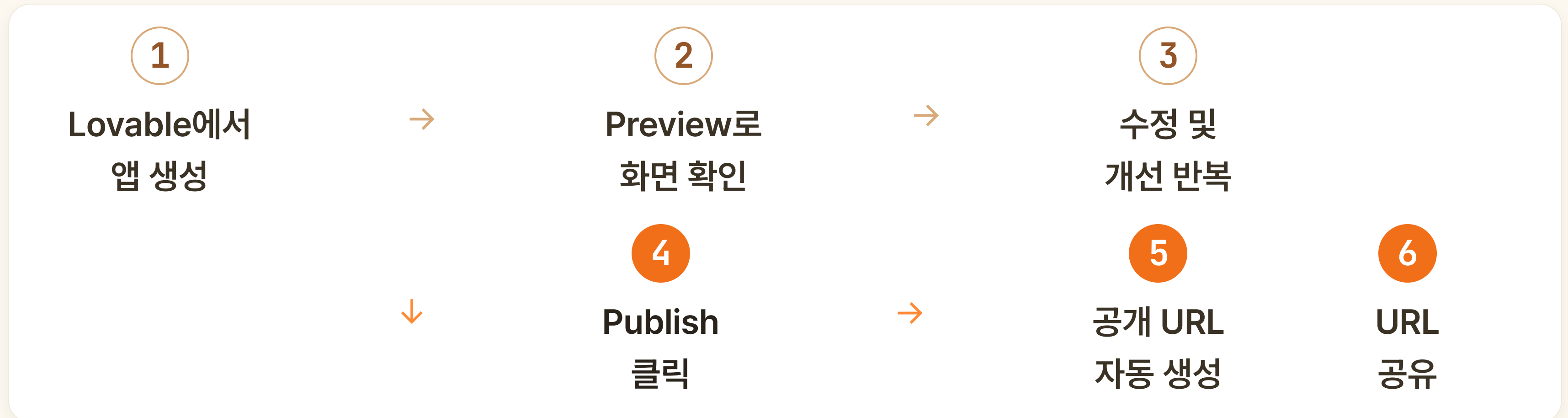
공개 URL

다른 사람이
접속할 수 있는 주소

문서를 **공유 링크**로 공개하듯, 앱은 **공개 URL**로 공개합니다.
배포는 결국 "앱의 공유 링크 만들기"에 가깝습니다.

LOVABLE에서는 어떻게 배포하나

Publish 버튼 하나면 공개 URL 생성.



GitHub 프로젝트 기록실

Publish 시연용 공개 무대

PUBLISH 누르기 전 체크리스트

공개 URL은 곧 “누군가 볼 수 있다”는 뜻입니다.

배포 전 5초 점검

- 1 실제 개인정보가 들어가 있지 않은가?
- 2 실명·사건명·내부자료가 남아있지 않은가?
- 3 API 키나 비밀번호가 노출되지 않았는가?
- 4 '가상 데이터', '공개용 데이터'인가?
- 5 공개 범위(누구까지 볼 수 있는지)를 생각했는가?

권장 화면 문구

“이 앱은 **연구모임 PoC 시연용**이며,
실제 개인정보·민감정보·내부자료를
입력하지 마세요.”

→ 첫 화면 상단 또는 입력창 옆에 작게 노출

PART 3

보안 — 배를수록 더 조심



복습 : API란?

API = 다른 서비스 기능을 빌려 쓰는 창구

예시 · 내 앱이 외부 AI에 요청

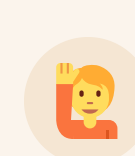
 "이 문서 요약해줘"

 "이 질문에 답변해줘"

 "이 주소 좌표 알려줘"

서비스끼리 이렇게 대화하는 방식이 **API**입니다.

한 줄 비유

 **내 앱**
= 손님

 **외부 서비스**
= 식당

 **API**
= 주문창구

모든 기능을 직접 만들지 않아도,
필요한 기능을 빌려 쓸 수 있습니다.

API 키란?

API 키 = 신분증 + 결제카드



신분증

누가 요청했는지 확인



결제카드

사용량에 따라 비용 연결



출입증

허가된 기능만 접근 가능

! 절대 하면 안 되는 것

× AI 채팅창에 실제 API 키 입력

× 코드에 실제 API 키 직접 작성

× GitHub에 API 키 업로드

.ENV / SECRETS를 쉽게

비밀정보는 **코드 밖 금고**에 둡니다.

문제

코드 안에 API 키를 쓰면,
GitHub 공개 시 함께 노출될 수 있습니다.

해결

비밀정보를 별도 보관함에 저장하고,
코드와는 분리합니다.

쉽게 이해하기



코드

= 공개 가능한 업무 매뉴얼



API 키

= 금고 비밀번호



.env / Secrets

= 비밀 보관함

AI에게 이렇게 말하세요

보안 유의사항은 자연어로도 충분히 지시할 수 있습니다.

× 나쁜 요청

"내 실제 API 키는 sk-abc123... 이거야. 연결해줘."

✓ 좋은 요청 ① 비밀번호 분리

"실제 API 키나 비밀번호는 채팅창이나 코드에 쓰지 말고,
내가 나중에 안전한 보관함에 직접 넣을 수 있게 만들어줘."

✓ 좋은 요청 ② GitHub 차단

"GitHub에 올라가면 안 되는 비밀번호 파일은 자동으로 제외되게 설정해줘."

GITHUB에 올라가면 안 되는 것

좋은 저장소지만, **모든 걸** 올리는 곳은 아닙니다.

✓ 올려도 되는 것

 앱 코드

 샘플 데이터

 설명 문서 (README 등)

× 올리면 안 되는 것

 API 키 · 비밀번호

 실제 개인정보

 내부자료 · 비공개 문서

`.gitignore` 업로드 **차단목록**

 **Lovable** "비밀정보 파일은 GitHub에 안 올라가게 막아줘."

보안 5초 체크리스트

보안은 마지막이 아니라 **매 단계의 습관**입니다.

STEP 1

프롬프트 입력 전

실제 개인정보를
넣고 있지 않은가?

STEP 2

GitHub 연결 전

API 키가 코드에
박혀 있지 않은가?

STEP 3

Publish 전

공개되어도 되는
데이터·화면인가?

STEP 4

공유 전

누구까지 볼 수
있는 링크인가?

**빠르게 만들되,
공개되면 안 되는 것은 절대 빠르게 공개하지 않는다.**



PART 4

시연발제 — 조직학습의 도구

다음 시연발제는 이렇게

결과물 그 자체보다 문제의식 · 과정 · 한계 · 시사점 공유

기본 발표 구조 6단계

① 어떤 업무 문제에서 시작했나?

② 무엇을 만들었나?

③ 어떻게 만들었나?

④ 더 잘 만들려면 무엇이 필요한가?

⑤ 어떤 한계를 느꼈나?

⑥ 조직에 어떤 시사점이 있었나?

★ 특히 중요한 발견

- 어떤 데이터가 부족 했는가?
- 어떤 데이터가 표준화 가 안 되어 있었는가?
- 앞으로 어떤 협업 이 필요하겠는가?
- 기존 업무 방식이 왜 AI 친화적이지 않았나 ?
- 향후 ISP / AX 사업 에서 무엇이 필요할까?

POC 하나가 조직에 알려주는 것

작은 PoC가 우리 조직의 **AX 준비상태**를 드러냅니다.

1

업무 적합성

어떤 업무가 AI의 도움을
받을 수 있는가?

2

위험 영역

어떤 업무가
아직 어렵거나 위험한가?

3

데이터 요구

어떤 데이터가 필요한가?

4

데이터 상태

정리·표준화가 되어 있는가?

5

협업 구조

부서/국/조직 간 협업 구조는
어떻게 바뀌어야 하는가?

6

AX·ISP 요구사항

향후 사업에서
무엇을 요구해야 하는가?

오늘의 결론

주소 · 기록 · 비밀, 그리고 학습.

01

주소

배포를 통해 다른 사람이 볼 수 있게 한다.

02

기록

GitHub로 변경이력과 협업기반을 남긴다.

03

비밀

API 키·개인정보·내부자료는 공개하지 않는다.

04

학습

PoC로 업무·데이터·협업구조 문제를 발견한다.

APPENDIX

오늘의 개발용어, 일상어로 번역하면?

Git	변경이력 자동 기록 시스템
GitHub	변경이력 보관하는 클라우드 협업 저장소
Repository	프로젝트 보관함
Commit	
Push	내 작업을 GitHub에 올리기
Pull	GitHub 최신 내용을 가져오기
Branch	원본을 건드리지 않는 별도 작업선
Merge	작업선을 원본에 합치기

PR	"합쳐도 될까요?" 검토 요청
Clone	프로젝트 전체 복사
배포	앱에 공개 주소 만들기
Publish	Lovable에서 공개 URL 생성
API	외부 기능을 빌려 쓰는 창구
API 키	신분증 + 결제카드
.env / Secrets	비밀정보 보관 금고
.gitignore	GitHub 업로드 차단목록